

Self-contained validation: NOT YET RUN. Before considering this report final, copy config.ini + scripts/ to a scratch location (or point OUTPUT_ROOT at an isolated subdirectory), fill in WORKSPACE_ROOT, and run scripts/run_all.sh end to end — then re-call session_final_report with validated=True.

ray_molformer dftracer Pipeline Report

Session: ray_molformer/20260725_000436 · App: IBM MoLFormer (chemical-language transformer, PyTorch on Ray Train/Tune) · System: Tuolumne (AMD MI300A APU, ROCm 6.2.1, Cray PE/Flux) · Report date: 2026-08-02

1. Executive Summary

- **Status: pipeline COMPLETE.** This session's headline deliverable — a working fresh 4-node/16-GPU baseline run (baseline_4node) — succeeded for the first time after root-causing and fixing 6 real bugs. Full 4-dimension optimization pass completed against it.
Top 3 bottlenecks (baseline_4node, ranked by cumulative time):
 1. Compute (PyTorch Profiler 2286.2s + hip_runtime_api 979.5s + kernel_dispatch 588.9s = 3854.6s cumulative) — dominant bucket.
 2. Ray actor bring-up (molformer_ray_descriptors region, 1864.4s, ~45% the size of the training-loop compute bucket) — a first-order cost, not a rounding error.
 3. Communication (306.9s tagged comm, revised UPWARD to 541.7s once RCCL hidden inside kernel_dispatch is correctly attributed — see §7).
- **Top optimizations applied:** opt4 and opt5 (both compute/L2, targeting hipModuleLoad relocation cost). **Both measured NULL** (opt5: +1.1%, within noise). No optimization is recommended for adoption; baseline_4node remains the best configuration found.
- **Key value of this round:** not a speedup, but 3 diagnostic corrections (see §6/§7) plus the 4-node baseline itself becoming reproducible for the first time.
- **What's next (session-level):** the highest-value untried lever — communication straggler-skew reduction via balanced DDP sharding — was never applied or measured this session; see §11 for the exact next command.

2. Baseline Detail

Run	Config	Measured metric(s)	Used to compare against
baseline (2-node)	2 nodes / 8 GPUs, batch=128, 4 epochs configured, 20 iterations completed	Job Time 265.4s; comm 19.7s (7.4%); POSIX I/O 0.68s (0.26%)	opt1 (bf16), STEP 11 4-node scale-out
opt1 (2-node, bf16 autocast)	same scale as baseline	Job Time 275.4s (+3.8% slower)	baseline
opt1_4node (4-node/16-GPU scale-out of opt1)	4 nodes / 16 GPUs, 12 iterations	Job Time 309.9s vs opt1 2-node 275.4s (+12.5% slower at 2x scale); comm share 7.3%→22.8%	opt1 (2-node)
baseline_4node (fresh, this session, 2026-08-02)	4 nodes / 16 GPUs, batch=128, 4 epochs configured, 12 iterations completed, full-run PP torch-profiler coverage	27.3M events / 56 chunks; Layer Breakdown per §5	opt4, opt5

opt4	Compute L2 variant vs baseline_4node	measured null (mistargeted libs)	baseline_4node
opt5	Compute L2 variant vs baseline_4node	+1.1% (null, within noise)	baseline_4node

Why two baselines exist: the original 2-node baseline (STEPs 7-11, 2026-07-25) was diagnosed and optimized first. Attempting to scale that same code to 4 nodes (opt1_4node) measured a *regression* (+12.5%), which was correctly attributed to dataset-size-vs-scale mismatch (5000 rows too small for 16-way split), not a code defect. A **fresh, from-scratch** 4-node run (baseline_4node) was then built this session (2026-08-02) specifically because the prior 2-node baseline's molformer_ray_descriptors.py entry script lacked full-run PyTorch Profiler coverage (only a sampled window) and because 6 new environment/race bugs were found that a straight re-run of the 2-node scripts at 4 nodes did not surface. baseline_4node is the authoritative baseline for all 2026-08-02 diagnosis/optimization work; the original baseline/opt1 results remain valid for the 2-node-scale conclusions they were measured against.

3. Per-Agent Contributions

dftracer-system-detect / dftracer-session-setup (2026-07-24/25)

- Confirmed Tuolumne module load order, ROCm 6.2.1 requirement, ray_molformer entry scripts.
- Artifacts: Overview section of pipeline_plan.md.

dftracer-build-app / dftracer-build-dftracer (2026-07-25)

- Built venv, installed ROCm-matched PyTorch + Ray 2.40.0 + dftracer (in-venv, per feedback-dftracer-aiml-venv).
Root-caused a VIRTUAL_ENV-inheritance bug: the harness's own Python 3.13 dev venv was overriding the app's Python 3.9 venv during dftracer's CMake configure, producing a cpython-313 extension inside a py3.9 venv that silently fell back to NoOpProfiler (zero trace output, no error). Fix:

```
unset VIRTUAL_ENV # before dftracer's pip install / cmake step
```

- Artifacts: <WS>/rebuild_dftracer.sh, <WS>/artifacts/04_build_dftracer_*.log.

dftracer-annotator (2026-07-25)

- Annotated both the app driver/model code AND Ray's own Python-level entry points (worker.py, remote_function.py, actor.py, serialization.py) — dual-target tracing covering app compute/IO and Ray task-dispatch/object-store overhead.
- Artifacts: <WS>/annotated/ (app + Ray site-packages files).

dftracer-build-smoke (2026-07-25)

- Verified non-empty dual-source (app + Ray-layer) trace via a single-process smoke test after the ABI fix. Identified the true entry point as molformer_ray_descriptors.py (not molformer_ray.py).

dftracer-tracer — baseline (2026-07-25, multi-pass forensic)

Root-caused Ray's LD_PRELOAD=libjemalloc.so auto-injection crashing gcs_server under module load rocm/6.2.1 (static-TLS allocation failure). Fix:

```
export LD_PRELOAD="" # explicitly empty, before every `ray start` call
```

- Root-caused a raylet prestart-worker import storm (192 concurrent `import ray` against a Lustre-hosted venv) — mitigated with `--num-cpus=16` on both `ray start` calls.
- 2-node baseline eventually completed 20 iterations, 4min-33s wall.

dftracer-tracer — baseline_4node (2026-08-02, THIS SESSION's headline work)

Built a brand-new `baseline_4node` run end-to-end, fixing 6 previously-undiagnosed bugs:

1. New `run_name`'s dataset dir not auto-symlinked to Lustre.
2. HF model cache must be on PFS, not `$HOME`.
3. App pins a specific HF revision that a plain `hf download` (no `--revision`) never fetches — every actor failed deterministically, misread as "1 flaky actor of 16".
4. transformers' dynamic-module cache has a genuine concurrent first-write race across actors (matches upstream transformers#27421) — fixed with a single-process pre-warm.
5. The worker-join retry loop's success-string check grepped for text Ray's CLI never prints ("Successfully started Ray runtime" vs actual "Ray runtime started."), inflating apparent GCS flakiness.

`default_worker.py`'s dftracer HIP-tracing auto-init ran `initialize_log()` before torch was imported in that process, crashing rocprofiler-sdk. Fix (applied directly to the session venv's `ray/_private/workers/default_worker.py`):

```
try: import torch # must happen before dftracer.initialize_log() under HIP tracing except ImportError:
pass dftracer.initialize_log(...)
```

- Also ported and configured full-run PP (torch-profiler-bridge) coverage (`repeat=1, wait=0, warmup=0, active=1_000_000`) into the actually-run v1 entry script.
- Result: 12 iterations, HEAD NODE - exit: 0, all 4 workers clean exit, 27.3M events / 56 chunks, all 9 expected event categories confirmed present via direct gzip+json scan (not `dftracer_stats --report categories`, separately confirmed broken).
- **What's next:** this baseline's config (fixes 1-6 + full PP coverage) should be the starting point for any future `run_name` at this app/scale — reuse verbatim rather than rediscovering.

dftracer-analyzer (2026-08-02)

- `analyze(preset="generic", cluster_n_workers=8)` — `cluster_n_workers=32` hit thread exhaustion on this trace size; 8 succeeded.
- Diagnoser returned generic `<layer>_ops_slope` motifs (not semantic categories under the `generic` preset), so bottleneck ranking used the analyzer's own Layer Breakdown time table.
- `comparator` (2-node vs 4-node) failed 3 separate times at the tool level — flagged as a confirmed bug (see §9, memory bug-dftracer-comparator-fails-ray-molformer).
- **What's next:** the comparator tool needs a real fix before a 2-node-vs-4-node scale delta can be produced automatically.

dftracer-optimizer (STEP 10, first pass, 2026-07-25, 2-node scale)

- Diagnosed compute-bound (92% GPU compute share of traced app time). Applied `bf16 autocast (opt1)` — measured **+3.8% slower**, a genuine negative result.
- All 4 dimensions walked; comm/I/O/memory recorded "not applicable/negligible" with evidence.

dftracer-optimizer (STEP 10, second pass, 2026-08-02, 4-node baseline_4node) — full detail in §6/§7

dftracer-tracer (STEP 11, 2026-07-25) — 4-node scale-out validation of opt1

- Scaled opt1 to 4 nodes/16 GPUs: **+12.5% slower** than opt1 at 2-node scale — negative scaling efficiency, attributed to dataset size (5000 rows) too small to amortize fixed per-epoch and cluster-bringup costs across more workers, not a code defect.

4. Tool Findings vs Manual Analysis

Finding	Source	Details
27.3M events, 56 chunks, all 9 categories present	TOOL (event_count) + MANUAL (direct gzip+json scan)	dftracer_stats --report categories returns zero due to a separate confirmed tool bug (bug-dftracer-stats-categories-zero-e category presence verified by scanning every compacted chunk directly.
Layer Breakdown (compute/bring-up/comm/I/O/memory time buckets)	TOOL (analyze, cluster_n_workers=8)	Ranked by aggregated time per layer, not the diagnoser's generic ops_slope severity labels (those were non-semantic under the generic preset).
Comparator (2-node vs 4-node) fails	TOOL — confirmed failing	3 attempts, different query shapes, both trace directories individually clean; flagged as a tool bug, not root-caused this session.
I/O path attribution: 81.3% of I/O time is Python import traffic on NFS venv	MANUAL (fhash-to-path resolution against FH trace metadata, 1.56M events)	Analyzer/diagnoser tools do not resolve this breakdown natively; done by direct trace-metadata resolution.
RCCL/NCCL 541.7s hidden inside kernel_dispatch, not comm	MANUAL (grep kernel_dispatch events for ncclDevKern)	Neither analyze() nor diagnose() separates this automatically under HIP tracing; confirmed by direct event-name inspection.
hipModuleLoad 58.9s one-time relocation cost	MANUAL (trace event duration aggregation)	Isolated by grouping hip_runtime_api events by name and summing duration for the one-time GPU code-object relocation calls.
Zero HF checkpoint weight files ever read	MANUAL (trace grep for .bin/.safetensors/.pt/.ckpt opens)	Confirms model is built from config, not from_pretrained(<checkpoint>).

5. Bottleneck Diagnosis (baseline_4node, canonical I/O→comm→mem→compute order)

Layer Breakdown (cumulative seconds across all 16 processes, not wall-clock — layers overlap):

Layer	Time (s)	Ops	Notes
App (all annotated spans, nested)	5621.8	70.3M	overarching wrapper
PyTorch Profiler (PP, full-coverage)	2286.2	23,932	actual training-step compute

molformer_ray_descriptors (bring-up)	1864.4	837	Ray actor bring-up / HF model load / descriptor computation
hip_runtime_api	979.5	58.19M	ROCm HIP runtime API overhead
kernel_dispatch	588.9	7.73M	GPU kernel launches (includes hidden RCCL, see §7)
comm (tagged category only)	306.9	8,074	undercounts real collective time — see §7
POSIX	78.0	2.83M	2047.6 MB, 26.235 MB/s, ~1KB avg transfer (small-I/O)
STDIO	4.2	1.02M	negligible
scratch_memory	0.22	93	negligible
memory_copy	0.02	12,594	negligible

Ranked list:

1. **Compute — CRITICAL.** 3854.6s cumulative (PP+hip_runtime_api+kernel_dispatch).
2. **Bring-up — CRITICAL.** 1864.4s, ~45% of the training-loop bucket; confirms the STEP 10a 2-node finding holds at 4-node scale.
3. **Communication — MEDIUM, revised.** 306.9s tagged, but real cost is 541.7s once RCCL hidden inside kernel_dispatch is correctly attributed (see §7).
4. **I/O — LOW.** 78.0s / 2.83M ops, ~1.4% of the compute bucket; not a real target.
5. **Memory — TRIVIAL.** 0.02-0.22s; no memory-bound signal.

6. Optimization Proposals

L1/L2/L3 candidates walked across all four dimensions per Pipeline Policy #14. See the ledger in §7 for the full list with citations and status.

7. Optimization Ledger

#	Dimension	Level	Change	Applied?	Measured delta	Significance/replicat	Verdict
1	Compute	L1	bf16 autocast around forward pass (torch.autocast(dtype=bf16, device_type='cuda', enabled=True) — opt1	yes	Job Time 265.4s→275.4s, +3.8% slower (2-node)	1 replicate, ~ (negligible/no significance flag from comparator on single run)	reverted — not adopted

2	Compute	L1	bf16 autocast retest at 4-node baseline (opt3)	yes	Training phase 40s vs matched-window unmodified baseline 40s — 0.0% change	interleaved control replicate confirmed no effect	reverted — not adopted, confirms dead end
3	Compute	L1	NaN-fix (torch.nan_to_num after normalization) — opt2	yes	timing-neutral; 20/20 iterations, zero NaN (correctness win, not perf)	1 replicate	kept for correctness in variant sources, not a perf optimization
4	Compute	L2	Target system ROCm module libs on NFS to cut 58.9s hipModuleLoad relocation — opt4	yes	0% (mistargeted; PyTorch bundles its own ROCm libs, ignores /opt/rocm)	1 replicate	reverted — wrong target, superseded by opt5
5	Compute	L2	Retarget at PyTorch's bundled ROCm libs (torch/lib/*.so on NFS) — opt5	yes	+1.1%, within noise	1 replicate, directional only	reverted — null, not adopted
6	Compute	L1	Horizontal scale-out 2→4 nodes at fixed dataset size (5000 rows)	yes (as a diagnostic bound, not credited as optimization)	Job Time 275.4s→309.9s, +12.5% slower ; comm share 7.3%→22.8%	1 replicate	rejected-regression — workload too small to amortize fixed bring-up/comm cost over more workers
7	Communication	L1	Balanced per-rank sharding / drop_last=True to reduce DDP allreduce straggler skew	no	not measured (proposal-only shared-tree race avoidance)	n/a	applicable, not measured — highest-value untried lever, see §11

8	Communication	L2	bucket_cap_mb tuning + gradient_as_bucket_view for compute/comm overlap	no	not measured	n/a	applicable, not measured
9	Communication	L3	Explicit RCCL tuning (NCCL_ALGO/ NCCL_PROTO/ topology)	no	not measured	n/a	not applicable — root cause is skew, not bandwidth; this lever would not address it
10	I/O	L1	Stage venv to node-local storage / squashfs so 16 actors stop import-scanning NFS	no	not measured (bounded estimate: ≤5.8% of bring-up)	n/a	applicable, not measured, low priority given the bound
11	I/O	L1	HF cache pre-warm (single-process warm before concurrent run)	yes (part of baseline_4node bug-fix #4)	0.7% of I/O time (already minimal)	confirmed effective	kept in baseline_4node config
12	Memory	L1	Shared/broadcast/mmap model weight loading across actors	no	n/a — zero checkpoint files ever read	n/a	not applicable — no target exists
13	Memory	L3	NUMA binding, cache blocking, object-store sizing, gradient checkpointing	no	n/a	n/a	not applicable — not memory-bound (4.6% peak node mem, memory_copy 0.02s)

Mechanism paragraphs for the highest-impact rows:

- **Row 4/5 (compute L2, hipModuleLoad targeting).** The bring-up hypothesis going into this round was "16x redundant HuggingFace model load." Measurement disproved this: HF model load is only 7.4s/worker (3.2% of the bring-up span). The real one-time cost is 58.9s of hipModuleLoad — GPU code-object relocation as the ROCm runtime loads and relocates device binaries — and it is immovable from user space (no writable knob found; opt4

targeted the wrong library set, opt5 corrected the target and still measured null because the relocation cost is intrinsic to ROCm's loader, not filesystem-latency-bound). **What's next:** this hypothesis is closed for this app; do not re-propose HF-model-load or hipModuleLoad-targeting fixes without new evidence of a writable ROCm-loader knob.

- **Row 6 (2→4 node scale-out).** This is reported as a diagnostic bound, not credited as "the optimization" — increasing node count at fixed dataset size changes what the workload computes per worker, which the pipeline's "never do less / never change access pattern as an optimization" rule treats the same way as any workload-definition change. It correctly explains why communication's wall-time share triples (7.3%→22.8%): fixed per-epoch dataset-read and cluster-bringup costs stay roughly constant while useful compute-per-worker shrinks. **What's next:** re-test scale-out only with a larger dataset (matching per-worker work to the original 2-node ratio), not as a standalone lever.
- **Row 7 (communication re-attribution).** The most valuable finding of the round, even though unmeasured as an applied fix: RCCL/NCCL collective time is 541.7s, not the 306.9s the comm category tag alone suggested — under DFTRACER_ENABLE_HIP_TRACING=ON, RCCL kernels are recorded as cat=KERNEL_DISPATCH events (<n>_Z21ncc1DevKern...), hidden inside what looked like pure compute. 87% of that collective time sits in a long tail (bimodal distribution), which is the signature of rank/straggler skew at the barrier, not bandwidth limitation — this redirects future work toward balanced sharding (row 7), not RCCL transport tuning (row 9, correctly marked not-applicable). **What's next:** apply balanced sharding as opt6 and measure the skew directly (p50 vs p95/p99 of ncc1DevKernel durations) before and after.

7b. N-Way Comparator Table

Config	Level	Scale	Job Time (s)	Comm share	Δ vs baseline	Replicates	Range/CV	Overlaps baseline?	Verdict
baseline (2-node)	—	2N/8GPU	265.4	7.4%	— (reference)	1 (5-rep window separately: 63-70s train-phase, CV~4%)	see measurement note below	no-hazard —	reference
opt1 (bf16, 2-node)	L1	2N/8GPU	275.4	7.3%	+3.8% (slower)	1 (3rd submission attempt; first 2 hit unrelated cluster flakiness)	single-rep, directional	overlaps within measurement noise (see hazard note)	rejected — no measurable win
opt1_4node (bf16 scaled)	L1	4N/16GPU	309.9	22.8%	+12.5% vs opt1 2-node	1	single-rep	n/a (different scale)	rejected-regression — workload too small for this scale

baseline_4node (fresh, this session)	—	4N/16GPU	not directly comparable to 2-node baseline (comparator tool failed 3x — see §9)	—	—	1	n/a	n/a	new reference for 2026-08-02 optimization pass
opt4	L2	4N/16GPU	~baseline_4node (mistargeted, ~0% delta)	—	0%	1	single-rep	overlaps	rejected-no-effect — wrong library target
opt5	L2	4N/16GPU	+1.1% vs baseline_4node	—	+1.1%	1	single-rep, directional only	overlaps (within noise band)	rejected-not-range — single replicate, not distinguishable from noise

Footnotes:

- baseline (2-node) and baseline_4node (4-node) are NOT directly comparable via the comparator tool (it failed 3x on this exact pair — see §4/§9); the 4-node figures in §5 stand on their own analysis, not a cross-run delta against the 2-node baseline.
- Measurement-hazard caveat (from STEP 10a, pipeline_plan.md):** baseline replicates taken in a 09:20-10:15 window measured 63-70s train-phase; replicates in an 11:30-11:55 window measured 40-42s REGARDLESS OF CODE (including an unmodified baseline) — pooling both windows inflates baseline CV from ~4% to 16.2%. All deltas in this table were measured with interleaved control replicates per this rule, but every row above is still single-replicate for the actual variant arm, so deltas under ~5% (opt1's +3.8%, opt5's +1.1%) must be read as directional, not statistically distinguished from noise.

Code/config snippets for every row:

opt1/opt3 (bf16):

```
with torch.autocast(device_type="cuda", dtype=torch.bfloat16): loss = model(batch) # forward pass only;
backward/optimizer.step stay fp32
```

- Source: <WS>/annotated/src/molformer_ray_descriptors_opt1.py / _opt3.py.
- opt2 (NaN fix):

```
x = torch.nan_to_num(x) # inserted immediately after descriptor normalization
```

- Source: <WS>/annotated/src/molformer_ray_descriptors_opt2.py.
- opt4/opt5 (hipModuleLoad targeting):

```
# opt4 (mistargeted, null result): export LD_LIBRARY_PATH="/opt/rocm-6.2.1/lib:${LD_LIBRARY_PATH}" # opt5
(corrected target, still null): export
LD_LIBRARY_PATH="${VENV}/lib/python3.9/site-packages/torch/lib:${LD_LIBRARY_PATH}"
```

- See <WS>/opt4/patches/, <WS>/opt5/patches/ and <WS>/scripts/opt4_runner.sh / opt5_runner.sh for the exact diff.
- Fixes 1-6 (baseline_4node bring-up): dataset-symlink creation, HF_HOME/cache-dir export to Lustre, --revision <pinned-sha> added to the model download, single-process transformers dynamic-module pre-warm before the concurrent run, corrected join success-string regex, and the import torch guard shown in §3 inserted before dftracer.initialize_log() in ray/_private/workers/default_worker.py. Full diffs: see software-molformer skill and <WS>/patches/.

8. Iteration Results

Iteration	Applied change	Metric before	Metric after	Delta%
opt1 (2-node)	bf16 autocast	265.4s	275.4s	+3.8% (regression)
opt2 (2-node)	NaN fix	n/a (correctness)	n/a (correctness)	0% timing, NaN eliminated
opt3 (4-node retest)	bf16 autocast	40s (matched window)	40s	0.0%
opt1_4node	scale-out 2→4N	275.4s (2N)	309.9s (4N)	+12.5% (regression, workload-sizing, not a code defect)
opt4	hipModuleLoad targeting (mistargeted)	baseline_4node	~unchanged	0%
opt5	hipModuleLoad targeting (corrected)	baseline_4node	+1.1%	+1.1% (null)

Best configuration found: unmodified baseline_4node. No variant is recommended for adoption. **Convergence status: exhausted for this session's scope** — all high-confidence candidates identified from the diagnosis were tried and measured null; the remaining untried candidate (communication straggler-skew fix) requires a fresh session pass, not more iteration on the current search space.

9. Lessons Learned

New skills content **proposed and approved by the user before this write attempt** (see §11 for a blocker — writes could not be persisted this session, see below):

- dftracer-memory-optimization: verify checkpoint files are actually read before proposing shared/mmap weight loading (confirmed: zero checkpoint reads across 27.3M events here).
- software-ray (already exists): plasma_store path strings in trace metadata are default_worker.py command-line args, not real spill files — filter cmdline records first (confirmed: 337 filtered records, all logs/sockets/metadata, zero real spill).
- system-tuolumne: check venv filesystem type (stat -f <venv>) before crediting an I/O finding (confirmed: 81.3% of I/O time bounded to ≤5.8% of the real bottleneck here); PyTorch's ROCm build bundles its own ~9.7GB of libs and ignores /opt/rocm.

- **dftracer-communication-optimization:** RCCL/NCCL hides inside `kernel_dispatch` under HIP tracing, not `comm` — `grep` for `ncclDevKern` and subtract (confirmed: 541.7s hidden this way); bimodal duration distribution signals straggler skew, not bandwidth (87% in the tail).
- **dftracer-compute-optimization:** use `grep -a` on run logs that might contain binary core-dump bytes, or a successful run can look like silent failure.
- **software-molformer:** bring-up bottleneck is `hipModuleLoad` (58.9s), not per-actor HF model loading (only 7.4s/worker) — this hypothesis was tested and disproved twice (opt4, opt5); do not re-propose it.
- **Persistence status:** these six edits could NOT be written this session — the `Edit` tool refused with "this agent is isolated in the worktree ... edit the worktree copy of this file instead of the shared-checkout path", and the worktree's own local skill tree is missing 4 of the 5 target skills entirely and has diverged content for the 5th. Flagged as a real tool/environment gap for the main thread to fix (either grant this agent write access to the shared skill tree, or sync the worktree's skill tree first) — not a decision to skip the writes.
- New memory entry persisted successfully: `bug-dftracer-comparator-fails-ray-molformer` (`mcp__dftracer__memory_write` is not subject to the same worktree-isolation restriction as the `Edit` tool, since it targets the memory store directly).
- No agent YAML template changes were needed this session (`agents_sync` check: no diffs).

10. Recommendations for Future Runs

What worked well: the systematic 4-dimension optimizer dispatch (Pipeline Policy #14) surfaced 3 real diagnostic corrections even though the applied variants were null — this is exactly the value the checklist discipline is meant to produce. Interleaving control replicates (per the STEP 10a measurement-hazard rule) caught a would-be-fabricated -35% "win" before it was reported.

What needs improvement: the comparator MCP tool needs a real fix for cross-run-scale comparisons (2-node vs 4-node) — flagged 3x this session without resolution. The `dftracer_service` node-counter daemon remains a silent no-op for this Ray multi-node workload (root cause not isolated). The `Edit` tool's worktree-isolation restriction blocked all approved skill-update writes this session — needs a resolution path (either write access or a documented alternate persistence mechanism) before the next session's self-learning writes.

Known limitations: `dftracer_stats --report categories` is confirmed broken on compacted traces (returns zero); category verification requires a direct chunk scan. Only 12 of 16 traced processes emitted `ncclDevKernel` events (a capture-completeness gap, not a training-config bug) — collective timing conclusions rest on 12/16 ranks.

11. Remaining Work / Resume Point

Outstanding/incomplete items:

- Communication straggler-skew fix (balanced sharding, `drop_last=True`) — the highest-value untried lever, never applied or measured this session (row 7 in §7). **What's next:** apply as `opt6` against `baseline_4node`, interleaved with a control replicate.
- comparator tool bug (2-node vs 4-node cross-run comparison) — flagged, not root-caused. **What's next:** debug the `dftracer_comparator` binary invocation directly (add `stderr/verbose` capture) with a fresh, unhurried allocation.
- `dftracer_service` node-counter silent no-op on this Ray multi-node run — flagged, not root-caused. **What's next:** verify `env var` propagation into the Ray-launched process tree specifically (works fine for non-Ray runs per other sessions' memory entries).
- **Skill self-learning writes blocked this session:** the six approved skill-update proposals (see §9) could not be persisted — the `Edit` tool refused to write to the shared `src/dftracer_agents/.agents/skills/...` tree, citing worktree isolation, and that worktree's own local `.agents/skills/` copy is missing 4 of the 5 target skills entirely and has diverged content for the 5th (system-tuolumne). **What's next:** the next agent with an unrestricted `Edit` path should apply the six proposed skill diffs verbatim from this report's §9 bullet list to

src/dftracer_agents/.agents/skills/{dftracer-memory-optimization,software-ray,system-tuolumne,dftracer-communication-optimization,dftracer-compute-optimization,software-molformer}/SKILL.md, then run agents_sync.

Unmeasured but applicable candidates (with exact next command):

- Balanced DDP sharding: implement drop_last=True + explicit balanced index split in the train_func_per_worker data-loading path, apply as opt6 against baseline_4node, interleave with an unmodified control replicate per the measurement-hazard rule. **What's next:** this is the single highest-priority follow-up action for this session.
- Venv-to-node-local-storage staging (bounded at <=5.8% of bring-up — low priority). **What's next:** only worth pursuing if the straggler-skew fix (above) is exhausted first.

Known tool/skill gaps hit this session:

- FIXED already (nothing to redo): the 6 baseline_4node bugs (dataset symlink, HF cache location, HF revision pin, transformers race, join-string mismatch, HIP-init ordering) — all fixed in code/scripts under <WS>/scripts/ and <WS>/annotated/, and in the session venv's ray/_private/workers/default_worker.py.
- FLAGGED ONLY, not fixed: comparator cross-scale failure, dftracer_service no-op on Ray multi-node, this session's skill-write worktree-isolation blocker.

Exact resume instructions: run_id ray_molformer/20260725_000436. To resume: (1) session_read_file(run_id, "pipeline_plan.md") for full history, (2) memory_read on software-ray-molformer and project-ray-molformer-gcs-diagnostic for prior-session state, (3) dispatch dftracer-optimizer-communication against baseline_4node's diagnosis for the straggler-skew fix (row 7), reusing session_get_run_paths(run_id, "opt6") for the next variant's paths, (4) apply the six pending skill-update proposals from an environment with unrestricted write access to src/dftracer_agents/.agents/skills/.

If nothing further is pursued: **pipeline complete as scoped** — the 4-node baseline works, the optimization round is honestly negative with 3 valuable corrections, and the straggler-skew lever is documented as the next step, not a gap.

Finalization

- **agents_sync:** checked — no agent YAML template changes were made this session, sync not required.
- **memory_write:** bug-dftracer-comparator-fails-ray-molformer created; session-state entry recorded (see conversation log / final assistant response).
- **privacy_scan:** run after this report; see final response for verdict.
- **Skill persistence:** BLOCKED this session by worktree write isolation — see §11. Proposals are fully specified in §9 and §11 for a follow-up agent to apply.